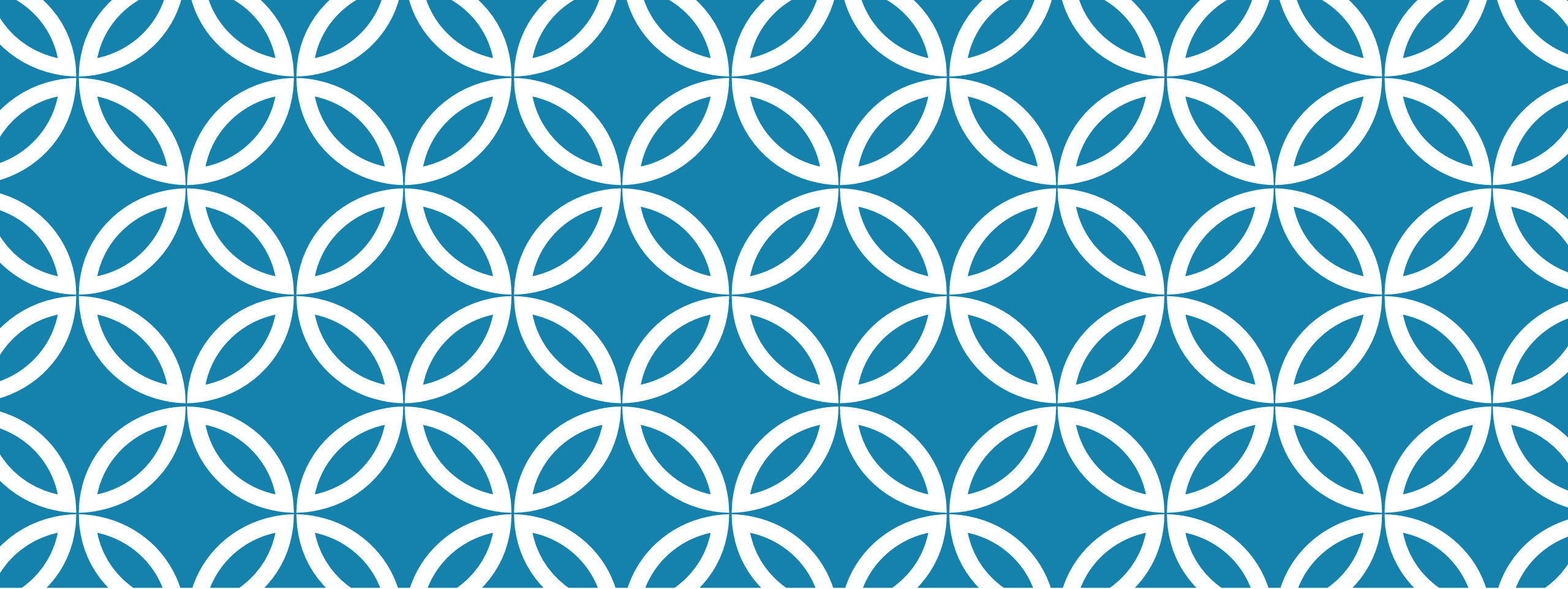




CSE455 - SOFTWARE TESTING

Chapter 3

PROCESS MODELS

Most software development projects are executed along lines of SDLC model that is chosen in advance

Divides a project into separate sections, phases, or iterations and arranges the resulting tasks and activities in a corresponding logical order

Model usually describe the: Roles that each task is assigned, project's participants responsible for each task

TYPES OF MODELS

Sequential development models

Iterative and Incremental development models

WATERFALL MODEL

Each development phase can only begin once the previous phase has been completed

Drawback: It bundles testing as a single activity at the end of the project

It means testing only takes place once all other development activities have been completed, and is thus seen as a kind of “final check” akin to inspecting goods that leave a factory

In this case, testing is not seen as an activity that takes place throughout the development process

V-MODEL

Development and testing are corresponding activities of equal value

Definition of requirements: This is where the customer and end-user requirements are collected, specified, and approved. The purpose and proposed features of the system are now stipulated

Functional design: The requirements are mapped to specific features and dialog flows.

Technical design: The functional design is mapped to a technical design that includes definition of the required interfaces to the outside world, and divides the system into easily manageable components that can be developed independently.

V-MODEL CONT..

Component specification: The task, behavior, internal construction, and interfaces to other components are defined for each component.

Programming: Each specified component is programmed (i.e., implemented as a module, unit, class etc.) using a specific programming language.

V-MODEL TESTING LEVELS

Component tests ensure that each individual component fulfills its specified requirements.

Integration tests ensure that groups of components interact as specified by the technical design.

System tests ensure that the system as a whole functions according to its specified requirements.

Acceptance test checks that the system as a whole adheres to the contractually agreed customer and end-user criteria

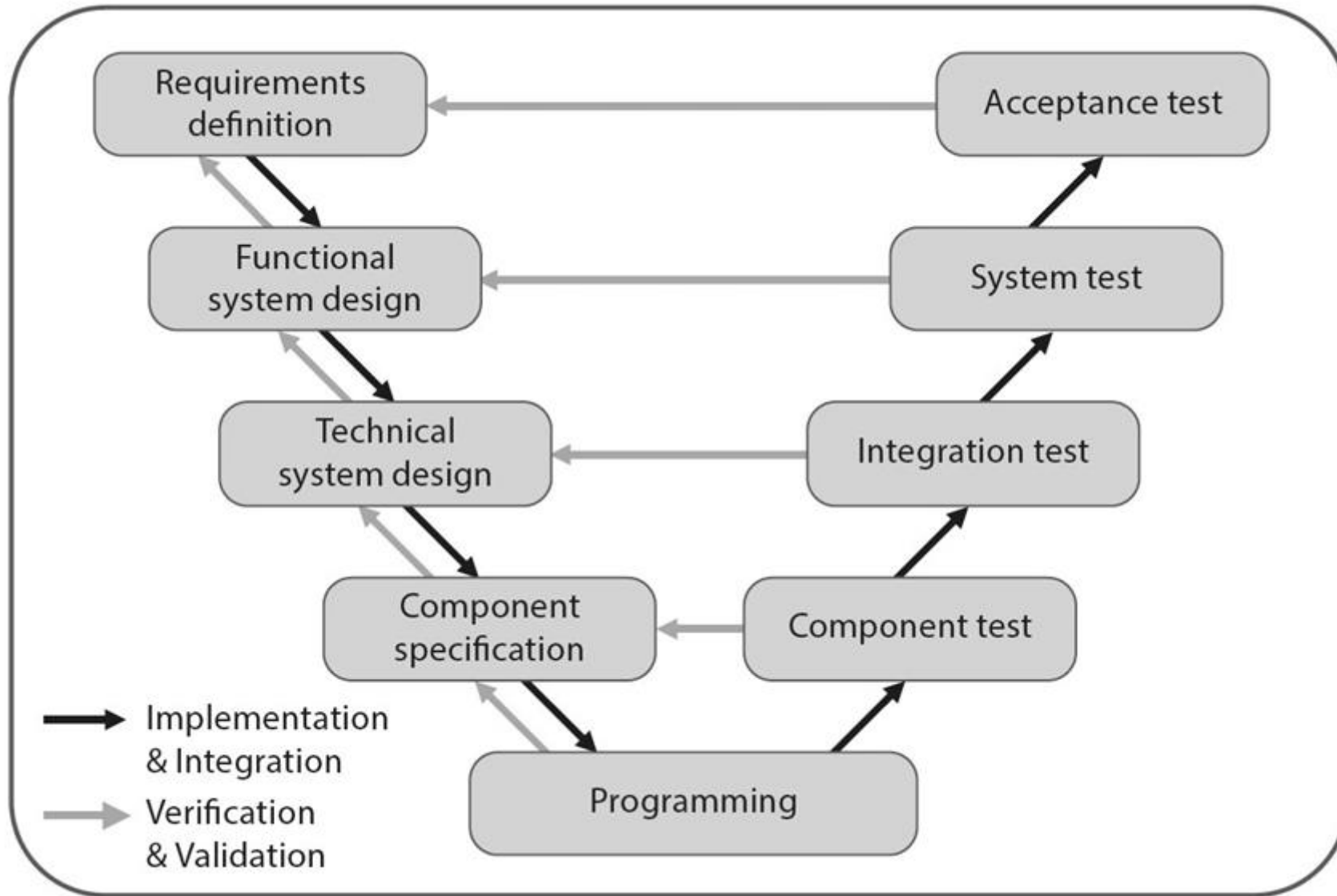
CONT..

Each test level checks the product at a different level of abstraction and follows different testing objectives, involve different testing techniques, different testing tools, and specialized personnel.

Each level is testing against its corresponding development level

Testing begins late in the development process, following implementation. This is wrong!

Testing must begin within the corresponding development step



PRACTICE

Component tests focused on verification or validation??

Acceptance tests focused on verification or validation??

ITERATIVE DEVELOPMENT

Use the experience from previous development stages with real-world and customer feedback from earlier system versions to improve the product in future iterations

Improve the product step by step in order to meet customer expectations

INCREMENTAL DEVELOPMENT

Develop a product in preplanned stages

Each completed stage offering a more full-featured version (increment)

Increments can vary greatly in size - for example from changing a simple web page layout to adding a complete new module with additional functionality

Objective is to minimize time to market - to release a simple product version to provide the customer as quickly as possible with a working version of the product or feature

Further enhancements will be then offered continually..

ITERATIVE INCREMENTAL DEVELOPMENT

Reduces the risk of developing a system that doesn't meet the customer's expectations

Examples – Spiral model, Rapid Application Development (RAD), Relational Unified Process (RUP)

All forms of agile software development are iterative-incremental development models – Extreme Programming (XP), Kanban, Scrum

Pace at which new increments/releases are created varies from model to model

CONT..

Each increment also requires new test cases that cover any additional functionality

Number of test cases you need to maintain and execute (on each release) increases over time

Test automation is an important tool when adapting your testing to agile development

CI/CD

Continuous Integration (CI) - When a component is modified, it is integrated into the previous complete build, followed by a fresh automated test run

Continuous Deployment (CD) - If the test run (during CI) is fault-free, the tested system is automatically copied to the production environment and installed there and thus deployed in a ready-to-run state

Continuous Integration + Continuous Deployment = Continuous Delivery

SOFTWARE DEVELOPMENT IN PROJECT AND PRODUCT CONTEXT

Appropriateness of a particular lifecycle model for the development of a specific product also depends on the contexts within which it is developed and used

The following project and product based factors play a role in deciding which model to use:

- The company's business priorities, project objectives, risk profile (time-to-market)
- Type of product – small project has less demanding development process as compare to large scale system
- Market conditions and technical conditions in which product is used
- Identified product risk – The safety aspect involved in designing and implementing a vehicle braking system
- Organization and cultural aspects – Difficulties generated by communication within international teams can make iterative or agile development more difficult

TAILORING

Development model be adapted and customized for use within a specific project

Integrating off-the-shelf commercial software into a larger system, interoperability tests at the integration testing stage

ATTRIBUTES OF GOOD TESTING

Testing activities are included as early as possible in the lifecycle

For every development activity, a corresponding test activity is planned and executed

Testers should participate early and continuously in this refinement process

TESTING LEVELS

During testing, a system has to be examined and tested on each level of its architecture, from the most elementary component right up to the complete, integrated system

COMPONENT TESTING

Checking the lowest-level components in a system's architecture

Alternative names for component are unit, module, class and corresponding tests are unit test, module test and class test respectively

Tests only a single component in isolation

Narrowly focused scope

Checks only internal functionality of component in question not its interaction with components external to it

Test basis – component specific requirements, component's design

Test object(s) – modules, units, classes, shell scripts, database scripts, data conversion and migration procedure, database content, configuration files

CONT..

Test Environment – module is called by passing required test data

Test Driver – Make call to required method, log test data and its corresponding result with timestamp

Developer tests – Component tests are usually prepared by developer of that component due to familiarity with constraints and acceptance criteria

Functional tests - Checking that the test object fully and correctly implements the functionality defined in its specifications

CONT..

Negative test - Serves the component under test with invalid input data instead of valid data and assume that the component will produce suitable exception handling as output. Robustness comes at a price

Efficiency test - How economically a component interacts with the available computing resources. This includes aspects such as memory use, processor use, or the time required to execute functions or algorithms

Maintainability test – How easy it is to enhance or extend a program. The main aspects to be tested are code structure, modularity, code commenting, comprehensibility and up-to-dateness of the documentation and so on. Only static testing can verify this test properly.

In order to check the completeness and correctness of the implementation, the component is subjected to a series of test cases, with each covering a specific combination of input and output data

TEST FIRST APPROACH

First design and automate your test cases and to program the code which implements the component as a second step

Test your code with the test cases you have already designed, and you then extend and improve your product code in small steps, repeating until the code fulfills your tests - *Test-driven development*

INTEGRATION TESTING

Assumes that the test objects handed over to this level are already component tested and that any component-internal defects have been corrected as far as possible

Developers, testers, and specialized integration teams then assemble groups of these components into larger units

Designed to find faults in the interfaces and the interaction between the integrated components

CONT..

Test basis – software architecture and design documents, interface specifications, workflow and sequence diagrams, use case diagrams

Software systems are nowadays rarely built from scratch, but are instead the result of the extension, modification, or combination of existing systems. Component testing doesn't usually include these kinds of existing or standardized objects, whereas integration testing has to cover these components and their interaction with other parts of the system

Integration testing often uses monitors – a program that keeps a check on the movement of data between components and logs what it sees

CONT..

Objective is to detect interface faults between components

Interface mismatches, missing files, or incorrect component divisions can cause build failures

Incorrect or unprocessable data causes crashes

Misinterpreted data leads to unexpected behavior

Timing issues affects performance and stability

CONT..

Skipping component testing and directly moving towards integration testing has certain drawbacks:

Many failures stems from individual components

Lack of direct access makes finding issues harder

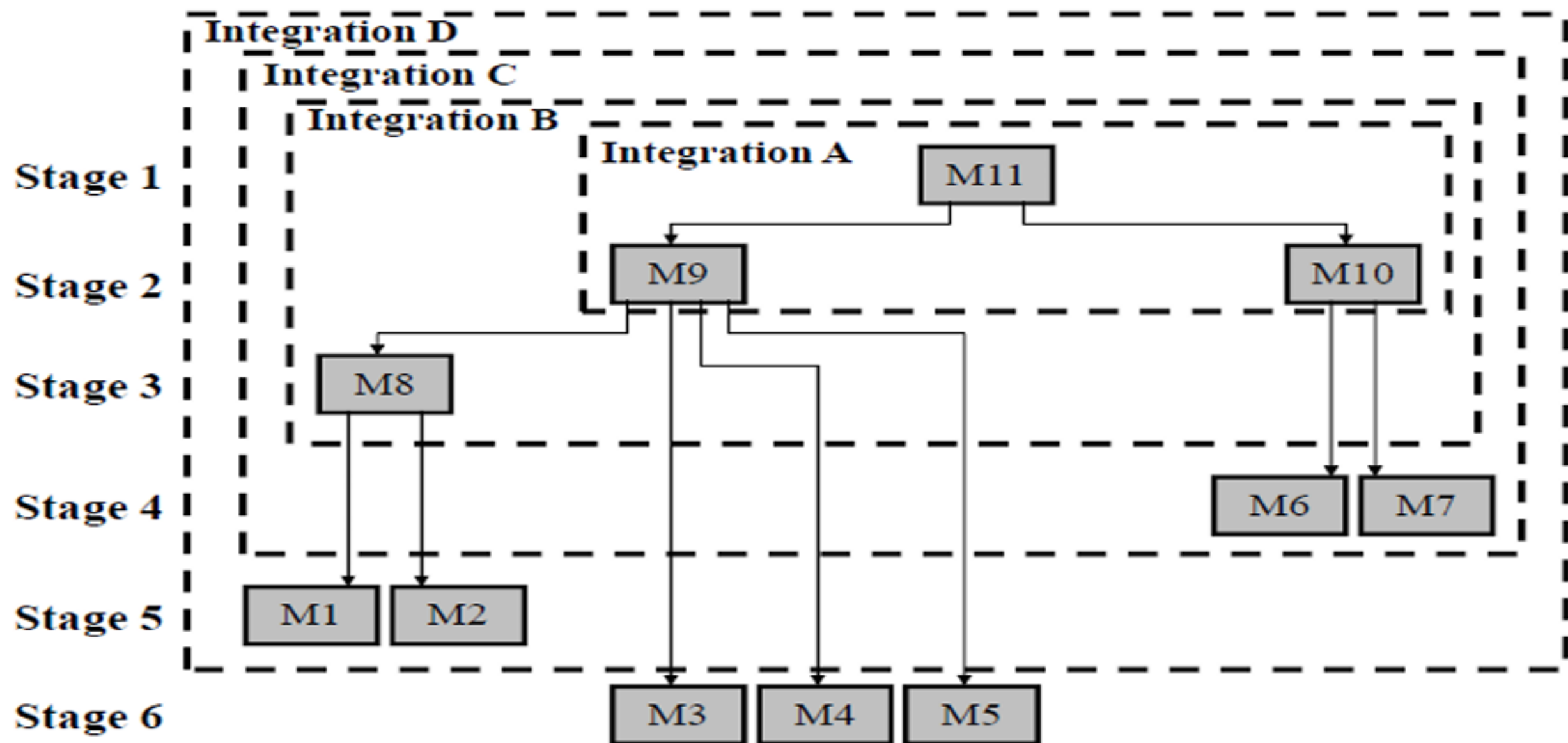
Some bugs remain undetected without isolated testing

TOP DOWN INTEGRATION

Starts with the main component (call others but isn't called itself)

Uses stubs to replace missing components

Top-down testing



ORDER OF TESTING INTERFACES FOR THE EXAMPLE HORIZONTALLY COVERED

- **Stage 1:** Unit tests of module 11
- **Stage 2:** Integration test A of module 11 integrated with modules 9 and 10, developed in the current stage
- **Stage 3:** Integration test B of A integrated with module 8, developed in the current stage
- **Stage 4:** Integration test C of B integrated with modules 6 and 7, developed in the current stage
- **Stage 5:** Integration test D of C integrated with modules 1 and 2, developed in the current stage
- **Stage 6:** System test of D integrated with modules 3, 4 and 5, developed in the current stage.

ORDER OF TESTING INTERFACES FOR THE EXAMPLE VERTICALLY COVERED

- **Stage 1:** Unit tests of module 11.
- **Stage 2:** Integration test A of the integration of module 11 with module 9, developed in the current stage.
- **Stage 3:** Integration test B of A with module 8, developed in the current stage.
- **Stage 4:** Integration test C of B with modules 1 and 2, developed in the current stage.
- **Stage 5:** Integration test D of C with module 10, developed in the current stage.
- **Stage 6:** Integration test E of integration D with modules 6 and 7, developed in the current stage.
- **Stage 7:** System test is performed after E has been integrated with modules 3, 4 and 5, developed in the current stage

BOTTOM UP INTEGRATION

Starts with basic components, larger units are built gradually and tested

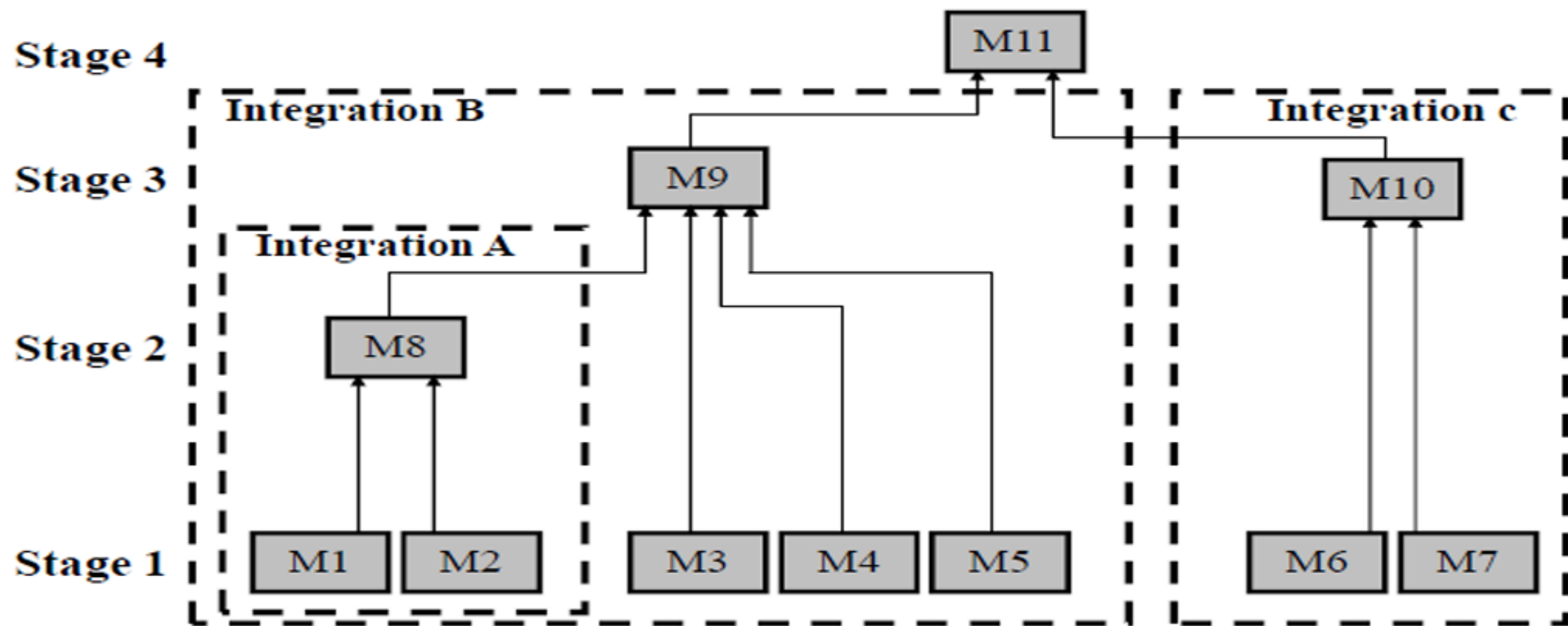
Requires test drivers for high level components

BOTTOM UP INTEGRATION CONT..

- Advantage of the bottom-up strategy is the relative ease of its performance, whereas the main disadvantage is the lateness at which the program as a whole can be observed (that is, at the stage following testing of the last module)
- The main advantage of the top-down strategy is the possibilities it offers to demonstrate the entire program functions shortly after activation of the upper-level modules has been completed
- In many cases, this characteristic allows for early identification of analysis and design errors related to algorithms, functional requirements, and the like.

BOTTOM UP INTEGRATION CONT..

- The main disadvantage of top down strategy is the relative difficulty of preparing the required stubs, which often require very complicated programming
- It is relatively difficult to analyze the results of the tests
- Top down integration leaves utility modules until the end, so that problems with these won't be detected early. If bottom-up integration is used then these might be found soon after integration testing begins.
- Top down integration also has disadvantage when the lower layers are finally added the upper layers may need to be retested



BOTTOM UP INTEGRATION CONT..

Stage 1: Unit tests of modules 1 to 7.

Stage 2: Integration test A of modules 1 and 2, developed and tested in stage 1, and integrated with module 8, developed in the current stage.

Stage 3: Two separate integration tests, B, on modules 3, 4, 5 and 8, integrated with module 9, and C, for modules 6 and 7, integrated with module 10.

Stage 4: System test is performed after B and C have been integrated with module 11, developed in the current stage

BI-DIRECTIONAL INTEGRATION

It is a combination of the top-down and bottom-up integration approaches used together to derive integration steps

A target layer is defined in the middle of the program and testing is carried out from the top and bottom layers to converge at this target layer.

However, it can be more complex to plan and selecting the best target layer can be difficult.

It is performed initially with the use of stubs and drivers

Stubs and drivers are software replacement simulators required for modules not available when performing a unit or integration test

Stubs provide down stream connectivity and stimulate the behavior of a missing component

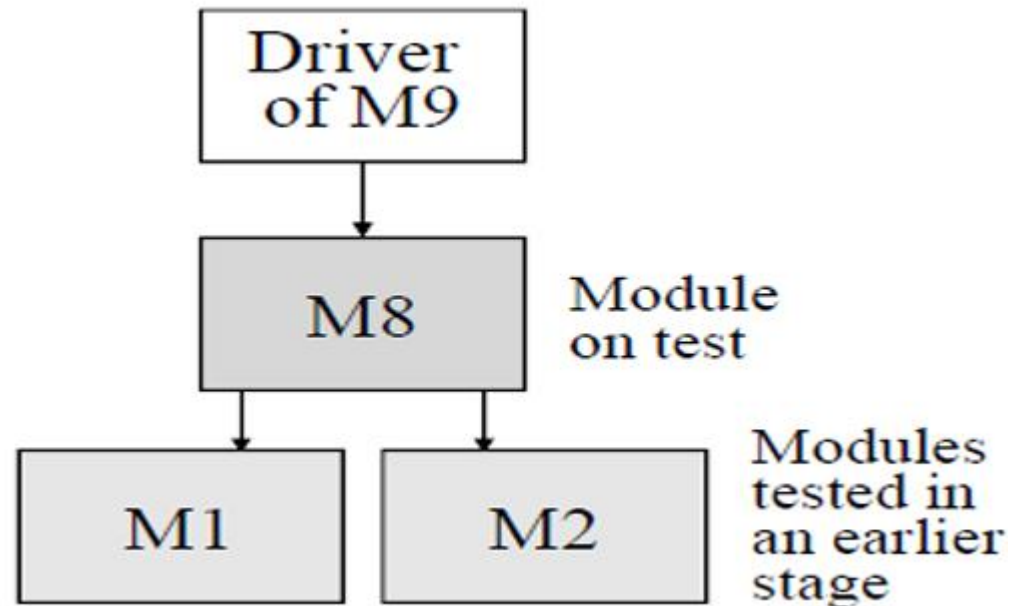
Stubs are required for top down testing of incomplete systems

BI-DIRECTIONAL INTEGRATION

- Drivers are used to provide upstream connectivity.
- A driver is a substitute module but of the upper level module that activates the
- module tested
- The driver is passing the test data on to the tested module and accepting the results
- calculated by it
- Required in bottom up testing until the upper level modules are developed

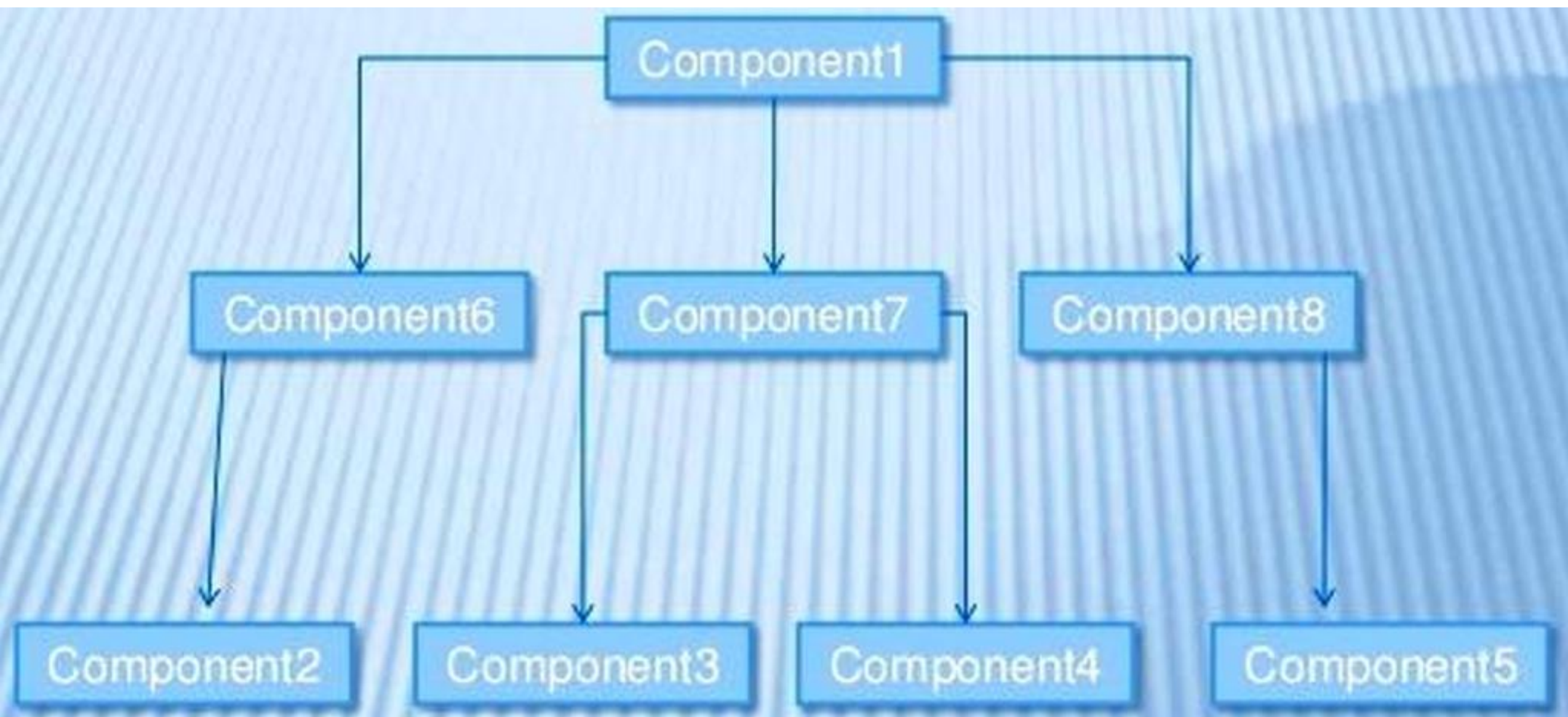
BI-DIRECTIONAL INTEGRATION

- At stage 2 lower level modules 1 and 2 are available; they have been tested and
- corrected at stage 1 of testing
- A driver is required to substitute for upper level module 9, which has not been completed



BI-DIRECTIONAL INTEGRATION

- After the functionality of these integrated components is tested, the drivers and
- Stubs are discarded
- This approach is also called *sandwich integration*



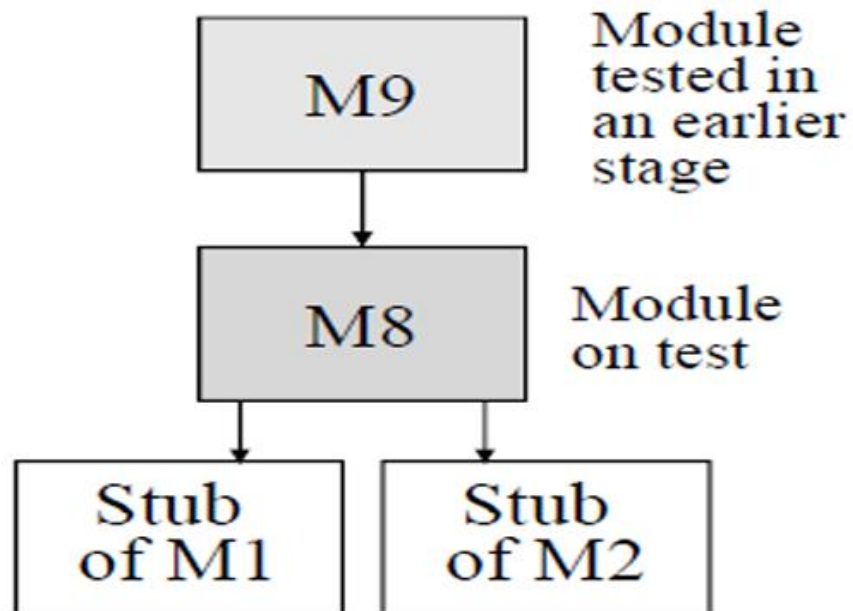
STEPS FOR INTEGRATION USING SANDWICH TESTING

Step	Interface Tested
1	2-6
2	3-4-7
3	5-8
4	(1-6-2)- (1-7-3-4)- (1-8-5)

TOP DOWN INTEGRATION (STAGE 3 OF PREVIOUS EXAMPLE)

Upper module 9 which activates module 8, is available; it has been tested and corrected at stage 2 of the testing.

Stubs are required to substitute for the subroutine level modules 1 and 2, which have not been completed



AD HOC INTEGRATION

Components are integrated randomly as they are completed

Saves time by integrating components as early as possible

Requires both stubs and drivers

BACKBONE INTEGRATION

A skeleton framework (backbone) is created and components are added progressively.

Continuous Integration (CI) follows this approach by adding new tested components to existing ones

Allows flexible integration in any way

Requires effort to create and maintain a backbone

AVOID BIG BANG INTEGRATION!!

All components are integrated at once without prior testing

Multiple defects appear simultaneously

No structured testing process

Wastes valuable testing time while waiting for all components

Difficult to identify and fix issues when everything breaks at once

System may not run at all, causing major delays

SYSTEM TESTING

Ensures the complete, integrated system meets specified requirements

Checks from customer and end-user perspectives

Identifies issues that emerge only at the system level

Avoid testing in production to prevent failures, crashes, and data loss

Requires complex environment setup

Verify functional & non-functional requirements

Identify missing, inconsistent, or undocumented requirements

Ensure database consistency, completeness, and accuracy

SYSTEM TESTING CONT..

- All the components of the system are integrated and tested as a single unit as per
- SRS
- System testing enables us to test, verify and validate both the business requirements and application architecture.
- Integration testing can be divided into
 - components or sub-system integration
 - final integration testing or system integration
- Complete system integration is covered as the last step
- This approach waits till all components arrive and one round of integration testing is done. This approach is called big-bang integration
- It reduces testing effort and removes duplication in testing

SYSTEM TESTING CONT..

Some disadvantages of system integration are

- When a failure or defect is encountered during system integration, it is very difficult to locate the problem
- The ownership for correcting the root cause of the defect may be a difficult issue to pinpoint
- When integration testing happens in the end, the pressure from approaching the release date is very high
- A certain component may take an excessive amount of time to be ready

SYSTEM TESTING CHALLENGES

Lack of clear documentation makes it hard for testers to verify expected behavior

- Testers struggle to define failures

Project stakeholders have different interpretations of system behavior

- Delayed decision making

Unclear requirements lead to a product that doesn't meet user expectations

- High chance of system failure, wasted development effort

Agile & iterative projects may have incomplete or evolving requirements

- More iterations than planned, potential project delays

CHOOSING INTEGRATION METHOD

Top-down approach is best suited for the waterfall and V

Bottom-up approach is best for iterative and agile methodologies

The selection of integration approach needs to consider availability of components, technology used, process, testing skills, and resource availability

Strategy chosen is actually determined in most cases by the developers' choice of development – not test strategy, that is, bottom-up or top-down.

Testers should follow the developers' approach because it is crucial that testing will be performed immediately after a module has been coded

Implementation of a testing strategy that differs from the development strategy will cause substantial delays in scheduling of the tests

ACCEPTANCE TESTING

Test takes place from the customer/end-user's viewpoint and is the only test that the customer is directly involved in or is actually responsible for

Variants

- User acceptance testing
- Acceptance by the system's operator
- Contractual and regulatory acceptance testing
- Field tests (i.e., alpha and beta tests)

CONT..

Test basis – user and system requirements, use cases, business processes, risk analysis, descriptions of the processes that use the system, forms, plus reports and descriptions of system maintenance and admin processes

Contractual acceptance test – Contract can be a formal agreement between two company internal departments that develop the product as joint product. Customer will decide whether or not to accept the delivered product.

Acceptance test criteria – detailed in the development contract will serve as test criteria, so these have to be clearly and unambiguously defined which demonstrate to the customer that contractual requirements have been fulfilled

In agile projects, continuous feedback is crucial. Acceptance tests should be included in every iteration

USER ACCEPTANCE TESTING

Ensures the system meets **end-user expectations**.

Critical when **customer** $\neq$ **end-user**.

Focuses on **usability, workflows, and real-world scenarios**.

Identify key user groups early.

Perform tailored acceptance tests for each group.

Tests should align with business processes and real-world scenarios.

ACCEPTANCE TESTING BY SYSTEM OPERATOR

Ensures system administrators can manage the system.

Key tests:

- Backup & recovery
- System restarts
- User administration
- Security & data handling

FIELD TESTING

Used for products deployed in multiple environments

Can't replicate all customer environments

Deploy pre-release software for testing in real-world settings



ALPHA TESTING

Conducted in-house by testers outside development teams



BETA TESTING

Performed at customer sites

Ensures the product works in real-world conditions



TEST TYPES

Functional and non-functional tests

Requirements and structure-based tests

FUNCTIONAL TESTS

Methods used to test the observable input/output behavior of a test object

Functional test cases are built using *black box* technique

Functional Suitability - How well a component is fulfilling its functional requirements

Functional test cases - Test that check functional attributes require at least one test case for each functional requirement

EXAMPLE

Functional requirements of the VSR-II *DreamCar* module:

- R100:** The user can choose a model for configuration from the current catalog
- R101:** The deliverable extras will be displayed for the selected model. The customer can select any of these to add to the configuration.
- R102:** The price for the current configuration (model plus extras) will be calculated according to the current price list and displayed in real time
- R103:** For the currently selected vehicle, only extras that are part of a deliverable configuration will be displayed. Non-deliverable extras for the current configuration will be grayed out.

EXAMPLE CONT..

Requirements based test cases against functional requirement 102 are:

R102.1: A vehicle model is selected; the price according to the sales manual is displayed

R102.2: An extra is selected; the vehicle price is increased by the price of this extra

R102.3: An extra is deselected; the vehicle price is reduced by the price of this extra

R102.4: Three extras are selected; these are discounted according to the specifications

EXAMPLE CONT..

Usually, more than one test case is required to test a functional requirement

In our example, Requirement 102 includes a whole series of price calculation variants that need to be covered by appropriate test cases (102.1 —102.4)

Functional testing takes place on every test level

NON-FUNCTIONAL TESTS

Non-functional requirements describe attributes of a system's functional behavior—i.e., how “well” a system or component should fulfill its function

Following non-functional characteristics should be covered and checked by corresponding tests

Load - Measurement of system behavior under increasing load (for example, number of parallel users, number of transactions)

Performance - Measurement of processing speed and response times for specific use cases, usually in conjunction with increasing load

Data volume - Observation of system behavior dependent on data volumes (for example, when processing very large files)

NON-FUNCTIONAL TESTS CONT..

Stress - Observation of system behavior in overload situations

Security - Combating unauthorized system and/or data access

Stability/Reliability - Under constant use (for example, by measuring the number of system malfunctions per hour for specific user profiles)

Robustness - When subjected to user errors, programming errors, hardware failure, and similar. Testing exception handling and restart/recovery behavior.

Compatibility/Data Conversion - Testing compatibility with existing systems, especially during data import/export

NON-FUNCTIONAL TESTS CONT..

Different Configurations - For example, using different OS versions, languages, or hardware platforms

Usability - Testing ease of learning and simplicity of use, including comprehensibility of output for various user groups

Compliance of system documentation with system behavior - For example, user manual vs. GUI, or configuration description vs. actual behavior

Maintainability - Comprehensibility and up-to-dateness of development documentation, modular structure, and so on

REQUIREMENT AND STRUCTURE BASED TESTING

Requirements-based testing is a black-box technique in which test cases are designed based on requirements. If component or integration tests are derived from technical specifications or requirements, these too can be classed as requirements based testing

Structure-based testing (structural testing, white-box technique) uses the internal structure/architecture of the software as its test basis. It analyzes things like the control flows within a component or the call hierarchy of procedures or menu structures

TESTING FOLLOWING SOFTWARE MAINTENANCE

Every software system needs to be modified and tweaked during its lifetime.

How does the testing process react to these changes?

Do all tests on every test level have to be repeated for each release?

Or is it sufficient to test only the elements that are directly affected by the changes that have been made?

MAINTENANCE TESTING

Confirmation testing involves repeating all test cases that revealed failures in the previous version. Such test cases have to be passed in order to classify the corresponding fault as corrected.

Simple local changes can cause unexpected and sometimes disastrous consequences and side effects in other parts of the system. How many and which types of test cases are necessary to reduce this risk has to be determined by a highly specific *impact analysis*

TESTING FOLLOWING RELEASE DEVELOPMENT

Most systems are subject to continuous development—for example, to build improved releases of the product. Such releases are usually coordinated with scheduled maintenance work

Testing following such release development follows two main objectives:

- ❑ Checking that every new or extended function works as intended
- ❑ Checking that pre-existing functionality is not (unintentionally) impaired

In order to achieve objective #1 you need to develop and perform new test cases. Objective #2 requires appropriate regression testing

REGRESSION TESTING

The process of repeating tests following changes to a program

Objective is to ensure that the parts of a revised system that haven't been changed still work as they used to before the changes took place

The simplest way to do this is to perform existing tests on the new version of the program

Test cases that are used in regression testing will be used regularly and often, and are therefore predestined for test automation which reduces cost of each test repetition

For features that need to be changed, the test cases need to be adapted too so that they cover the new functionality

COMPLETE VS PARTIAL REGRESSION TESTING

In practice, complete regression testing that runs all existing tests is usually too costly and takes too much time, especially when manual tests are involved

Repeat only the tests that were given high priority in the test schedule

Limit testing to certain specific configurations (for example, test only the English-language version, test only for a specific operating system, and similar)

This requires compromise between minimizing costs and accepting business risks

SUMMARY

Software development lifecycle models structure the software development process in sections, phases, or iterations. The two basic model types are “sequential” and “iterative/incremental”.

Sequential development models are characterized by development activities that are conducted in a linear (i.e., sequential) fashion.

Iterative/incremental models produce regular extended and/or improved product releases that enable timely feedback from the customer and the system’s users. This approach reduces time to market and also reduces the risk of developing a product that doesn’t fulfill the customer’s expectations. All agile development methods are classed as iterative/incremental.

SUMMARY

The sooner a defect is discovered, the less it will cost to rectify it. The V-model therefore requires verification procedures (such as reviews) at the end of every development phase. This helps to prevent the spread of subsequent defects.

Maintenance and incremental development continuously create new software product versions throughout the product's lifecycle. Each new version has to be tested in its own right, and the scope of such regression testing depends on the results of corresponding risk analysis